

COMP 5111 (2026Spring)

Assignment 1 Introduction & Soot Tutorial

Please download the materials (slide & example project) on course website if you haven't.

TA: Songqiang CHEN

Contact TAs

Name: CHEN, Songqiang
LU, Weiqi

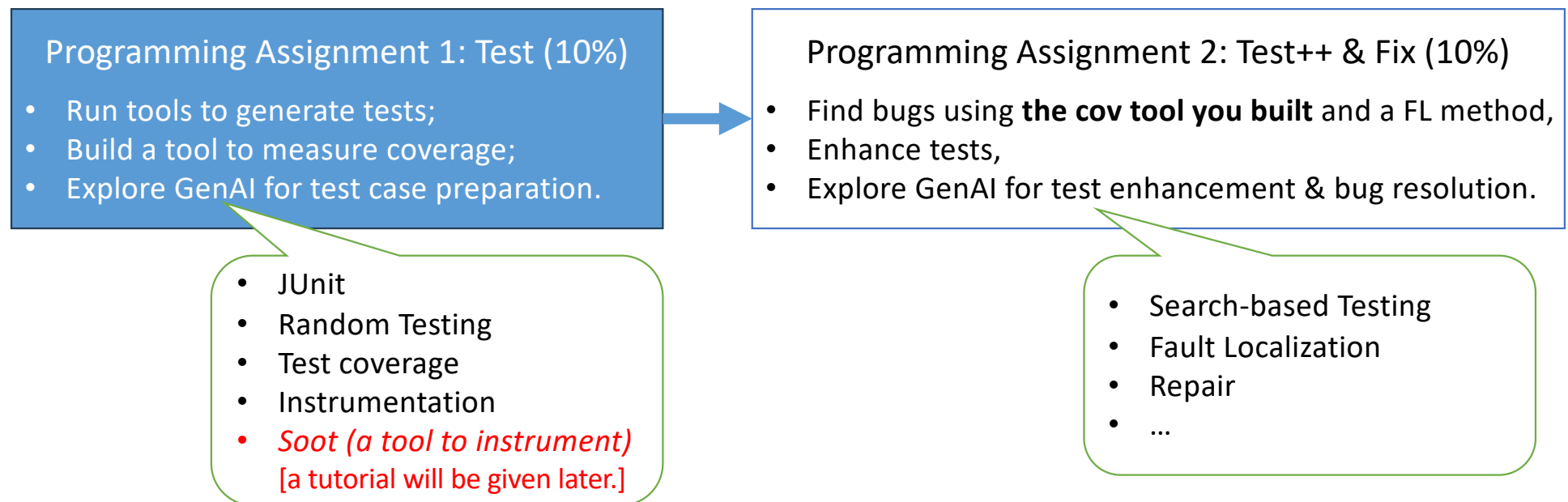
Email: csta5111@cse.ust.hk

Office Hour: Rm 3663 (via Rm3661)
Tue & Thu 11:50 - 12:20

Zoom: Please send us an email to
schedule a date and timeslot.

Programming Assignments of COMP5111 26S (20%)

To test and fix a Java program in "Subject.java" (Subject Class, Class Under Test, CUT).



Programming Assignment Environment

- **OS: Linux / MacOS / Windows**
 - Linux is highly recommended.
 - You can use the machine at CS lab.
- **JDK 11**
- **Eclipse 2025.12**
- **JUnit 4.12**, with hamcrest-1.3
- **Randoop 4.3.1**
- **Soot 4.2.1**

(May not be the latest, but should be stable and well compatible.)

Programming Assignment 1 Overview (10 pts)

- **Tasks of Test Preparation:**

- Task 1: Run Tools to Generate Test Cases and Measure Code Coverage (25%)
 - Task 1.1: To learn how to use **Randoop** for unit test generation
 - Task 1.2: To learn how to use **EclEmma in Eclipse** for code coverage measurement.
- Task 2: Line coverage measurement using **Soot** (40%) [Soot Tutorial]
- Task 3: Exploring the usefulness of **Gen AI** for specification-based testing (35%)

- **Submission:** Canvas, by 15 March 2026 (23:55)

Detailed Instructions: <https://github.com/CastleLab/COMP5111-2026Spring-Assignments-Students>

Task 1 (25%): Generate Test Cases & Measure Code Coverage

- Basically, task 1 only asks you to **run existing software** to generate tests and measure coverage.
- The most things you need to do is following the instructions in our repository. Some requirements may need your exploration of certain advanced parameters.
- Most steps are quite easy. Hints:
 - Make sure that you **correctly install** any related software and **set appropriate** environment variables and parameter values.
 - Some students get error: *Class Not Found* – Check your Java classpath setup.

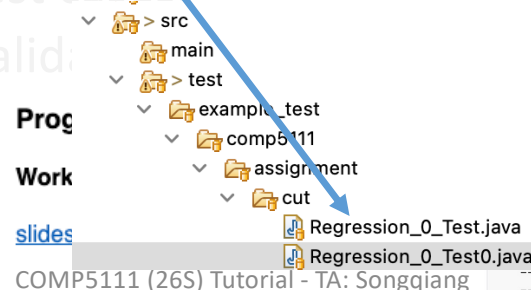
Task 1.1: Generate Tests with Randoop

- Objective:
 - Use Randoop to generate **FIVE regression test suites** for the program under test (named Subject.java).

- Tutorials: Available in our repository.
An example script is given in the example project.

- Steps:

- Run Randoop to yield test cases
- Run the test cases for valid



Random Testing with Randoop

Introduction

In this tutorial, you will learn how to use the random testing tool Randoop. Randoop is an automatic unit test generator for Java programs. It generates two types of test suites:

- Error-revealing tests that indicates errors in the Java code.
- Regression tests that capture current behavior of the program.

Steps

```
1 package comp5111.assignment.cut;
2
3 import org.junit.FixMethodOrder;
4
5 @FixMethodOrder(MethodSorters.NAME_ASCENDING)
6 public class Regression_Test0 {
7
8     public static boolean debug = false;
9
10    @Test
11    public void test01() throws Throwable {
12        if (debug)
13            System.out.format("%n%s%n", "Regression_Test0.test01");
14        char char0 = comp5111.assignment.cut.Subject.CharTasks.AUL;
15        org.junit.Assert.assertTrue("'" + char0 + "' != '" + "\000" + "'", char0 == '\000');
16    }
17
18    @Test
19    public void test02() throws Throwable {
20        if (debug)
21            System.out.format("%n%s%n", "Regression_Test0.test02");
22        int int0 = comp5111.assignment.cut.Subject.StringTasks.INDEX_NOT_FOUND;
23        org.junit.Assert.assertTrue("'" + int0 + "' != '" + (-1) + "'", int0 == (-1));
24    }
25
26    @Test
27    public void test03() throws Throwable {
28        if (debug)
29            System.out.format("%n%s%n", "Regression_Test0.test03");
30        boolean boolean1 = comp5111.assignment.cut.Subject.NumberTasks.isParsable("");
31        org.junit.Assert.assertTrue("'" + boolean1 + "' != '" + false + "'", boolean1 == false);
32    }
33
34    @Test
35    public void test04() throws Throwable {
36        if (debug)
37            System.out.format("%n%s%n", "Regression_Test0.test04");
38        java.lang.String str1 = comp5111.assignment.cut.Subject.StringTasks.chop("hi!");
39        org.junit.Assert.assertEquals("'" + str1 + "' != '" + "hi" + "'", str1, "hi");
40    }
41
42    @Test
43    public void test05() throws Throwable {
44        if (debug)
45            System.out.format("%n%s%n", "Regression_Test0.test05");
46        java.lang.String str1 = comp5111.assignment.cut.Subject.FilenameTasks.getName("hi!");
47        org.junit.Assert.assertEquals("'" + str1 + "' != '" + "hi" + "'", str1, "hi!");
48    }
49
50    }
51
```

Task 1.2: Measuring Test Coverage using EclEmma (in Eclipse)

- Objective:
 - Use EclEmma to measure the coverage of generated test cases.
- Tutorials: Avail in our repository.
- Steps:
 - Install EclEmma / Eclipse.
 - Create/Import the Java project.
 - Run the unit test cases.
 - Read and check the report.

Example screenshot (some notes):

Test coverage measurement using EclEmma

Introduction

In this tutorial, you will learn how to use the toolkit EclEmma in Eclipse to measure test coverage. (Please note that some screenshots and demo video are materials of last year's course.)

Steps

You can follow these steps to build the JAVA project on Eclipse, then use it to generate test coverage report.

Step 0: Install Eclipse 2023-12 and EclEmma

You can install the Eclipse 2023-12 following the [official website](#). If you have already installed Eclipse, you can upgrade your Eclipse through [this link](#).

NOTE: EclEmma is already in Eclipse 2023-12 by default. You may skip this step.

Step 1: Another common issue: Coverage = 0?

As you have installed Eclipse, you can find the Coverage As menu item in the Run menu. Clicking it will open the Coverage Configurations dialog box.

Step 1-1: Coverage Configurations

Clicking the Coverage Configurations... button will open the Coverage Configurations dialog box. In the dialog box, you can select the test suite to measure the coverage of. In the example, the test suite is Regression_6_Test (1).

Regression_6_Test (1) 2026年2月8日 下午3:44:03

Element	Coverage	Covered Branches	Missed Branches	Total Branches
comp5111.assignment.cut	20.5 %	218	848	1,066
Subject.java	40.9 %	218	315	533
Subject	0 %	0	0	0
DateTimeAlgorithms	74.1 %	103	36	139
GameConfiguration	4.5 %	5	107	112
GamePlace	37.5 %	6	10	16
GamePlayer	66.7 %	4	2	6
StringAlgorithms	38.5 %	100	160	260

show the name of test suite.

show the coverage of CUT and all inner classes. (NOT test suite.)

Task 1 (25%): Generate Test Cases & Measure Code Coverage (cont.)

- **Submissions:**

- Task 1.1: 5 regression test suites generated by you.

- Task 1.2: 10 screenshots.

For each regression test suite, one for line coverage and one for branch coverage.

- **Requirements:** -- *failure in these points cannot earn scores.*

- **T1.1:** Submit the **regression tests** generated by Randoop, but not the **error-revealing tests**.

- **T1.2:** Each test suite must achieve **$\geq 50\%$ line coverage** & **$\geq 40\%$ branch coverage** (reported by EcEmma)

- **T2.1:** The coverage is measured on the Subject class but not the tests (all previous notes).

- **T2.2:** The coverage shown in the screenshot should be measured by Eclipse, not other IDEs.

- **More Details on Submission Content, Format Requirement, Grading Scheme, etc.:**

(All are concretely explained in the detailed instruction in our repository.)

Task 2 (40%): Measuring Test Coverage with Soot

- **Objective:** Develop a coverage measurement tool with Soot!
- Sounds challenging?
It is like building blocks - once you understand idea of instrumentation, and how to instrumenting code using Soot – we will learn both today.

Task 2 (40%) – Background Knowledge

- What to measure?

(P)review of the related coverage: (a rough recap :)

- Line Coverage: the % lines that have been **executed** among all lines in class under test **on a set of specified test cases**.

$$\text{Coverage} = \frac{\# \text{executed lines}}{\# \text{total lines}}$$

- In general, not consider the “empty” lines without statements.
- Why do we care about it?
 - It reflects the test adequacy. A bug will never be detected unless it is executed (tested) at least once. (?)

Executed Tests

```
@Test
public void test1() {Subject.StringAlgorithms.startsWithIgnoreCase("1", "12");}
@Test
public void test2() {Subject.StringAlgorithms.startsWithIgnoreCase("12", "1");}
```

Covered **5** lines (with statements) in method; missed 4 lines

```
20 public static boolean startsWithIgnoreCase(String str, String prefix) {
21     if (str == null || prefix == null) {
22         return false;
23     }
24     if (str.length() < prefix.length()) {
25         return false;
26     }
27     if (str.startsWith(prefix)) {
28         return true;
29     }
30     String lcStr = str.substring(0, prefix.length()).toLowerCase();
31     String lcPrefix = prefix.toLowerCase();
32     return lcStr.equals(lcPrefix);
33 }
34
```

Coverage x				
Problems Javadoc Declaration Search Console Progress Terminal				
ample (2026年2月8日 下午4:10:16)				
ment	Coverage	Covered Lines	Missed Lines	Total Lines
startsWithIgnoreCase(String, String)	55.6 %	5	4	9

Task 2 (40%) – Background Knowledge (cont.)

- **Measure through what code/statements?**

We use **Jimple 3-Address Code**, the Java intermediate code used by Soot, in our PA.

- Instead of directly changing *Java src/bin code*, we **process the *Jimple 3AC statements*** and **count Java source lines associated with them**. – to realize coverage measure using Soot.

Java:

```
...  
(1) for (i = 0; i < 10; ++i){  
(2)     b[i] = i * i;  
(3) }  
...
```



Jimple Stmts:

```
    t1 := 0           (1) ; initialize i  
L1: if t1 >= 10 goto L2 (1) ; conditional jump  
    t2 := t1 * t1      (2) ; square of i  
    t3 := t1 * 4        (2) ; word-align address  
    t4 := b + t3        (2) ; address to store i*i  
    *t4 := t2           (2) ; store through pointer  
    t1 := t1 + 1        (1) ; increase i  
    goto L1             (1) ; repeat loop  
L2:
```

Task 2 (40%) – Background Knowledge (cont.)

- **How to measure?** (Methodology, not implementation)

Instrument: Insert some statements into the target program, in order to conduct additional operations to monitor program, while not changing the original code behavior.

- To answer “how many lines are executed”, we can edit the code so that:
 - Before running CUT, **we insert some code** to let machine auto record which stmt & line have been executed, and output the record at the end of execution.

```
t1 := 0           (1)
L1: if t1 >= 10 goto L2 (1)
    t2 := t1 * t1   (2)
    t3 := t1 * 4    (2)
    t4 := b + t3    (2)
    *t4 := t2       (2)
    t1 := t1 + 1    (1)
    goto L1         (1)
L2:
```

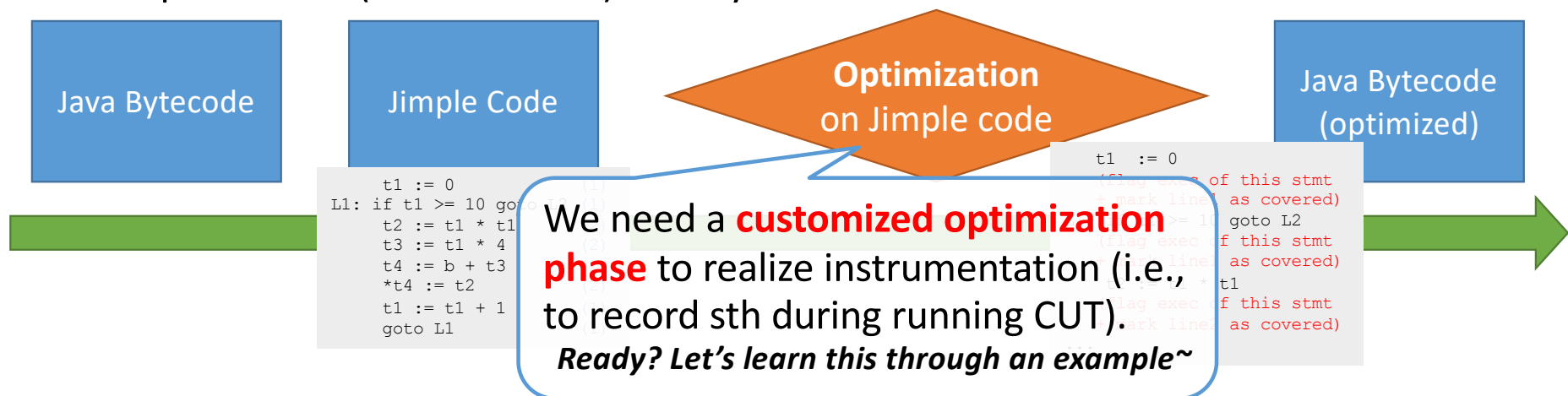


```
t1 := 0
(flag exec of this stmt
+ mark line1 as covered)
L1: if t1 >= 10 goto L2
(flag exec of this stmt
+ mark line1 as covered)
    t2 := t1 * t1
(flag exec of this stmt
+ mark line2 as covered)
    ...
```

- Then, during running CUT, the CUT with inserted code **remains the original functionality**, but it **will further** automatically **record** and **output** the runtime information.

Task 2 (40%) – Soot

- We use a tool called **Soot** to instrument program automatically.
 - A static analysis tool for Java code.
For **instrumentation**? -- automatically edit the code when traversing each code unit.
 - Input we use: **Java Bytecode**.
 - Intermediate Representation we use: **Jimple**.
 - Output: Edited (instrumented) Java Bytecode.



Task 2 (40%) – Example: Count Executed Returns using Soot

- **HOW?** – An example project to illustrate how to instrument.
(download the project, and take notes)

Coursework

Programming Assignments

Workshop: Assignment 1 Introduction

slides [soot example project](#)

- **Example Proj:** Count how many times the return stmt in static / instance methods in CUT are invoked by a test suite, respectively.

```
public class Subject {  
    public static int staticMethod_Add(int x, int y) { return x + y; }  
    public static double staticMethod_Add(double x, double y) { return x + y; }  
    public static int staticMethod_Mul(int x, int y) { return x * y; }  
    public static double staticMethod_Mul(double x, double y) { return x * y; }  
    public int instanceMethod_Add(int x, int y) { return x + y; }  
    public double instanceMethod_Add(double x, double y) { return x + y; }  
    public int instanceMethod_Mul(int x, int y) { return x * y; }  
    public double instanceMethod_Mul(double x, double y) { return x * y; }  
}  
  
void main() {  
    new Subject().staticMethod_Add(1, 2);  
    new Subject().instanceMethod_Add(1, 2);  
    new Subject().staticMethod_Add(1, 2);  
    // 2, 1  
}
```

- First of all, consider **insert what code to where with what utilities?**
 - Sol: **Insert** a statement to **increase a counter before each return stmt.**
- Then, implement to realize the transformation.

Task 2 (40%) – Example Use of Soot

How to realize: Insert a statement to **increase a counter before each return stmt?**

- Implement another set of Java source codes as our tool.
- These codes consist our tool. They are NOT a part of the program to test (Subject.java).
- **What do we need? Three components:**
 - **A counter class** maintaining several variables / functions to record the counts (e.g., a **var** counting how many times a static return stmt is invoked; a **func** to increase this var; ...)
 - **An instrumentation class (a customized transformation)** that defines what to do when traversing a code snippet (e.g., to invoke the **func** to increase the **var** before each return)
 - **An entry point for the whole tool** to setup soot and start the transformation.

Follow?

Task 2 (40%) – Example Use of Soot – Our Own Phase – Counter

Implement: insert **what** code to **where**, with **what utilities**

- The helper class for recording data: **Counter**

```
1 package castle.comp5111.example;
2
3 public class Counter {
4     private static int numStaticInvocations = 0;
5     private static int numInstanceInvocations = 0;
6
7     public static void addStaticInvocation(int n) { numStaticInvocations += n; }
10
11     public static void addInstanceInvocation(int n) {
12         numInstanceInvocations += n;
13     }
14
15     public static int getNumInstanceInvocations() { return numInstanceInvocations; }
18
19     public static int getNumStaticInvocations() { return numStaticInvocations; }
22 }
```

Counter variables to record what we count

How to realize this?

The function to call (invocation inserted to CUT) to increase private counter variables.

The function to call after executing the instrumented CUT to print counting results.

```
package castle.comp5111.example;
1
2 // This class is the subject in our tutorial of soot.
3 // We first generate tests using random.
4 // Then, we will use soot to instrument this class and count the total number of static/instance methods are called in tests.
5
6 public class Subject {
7     public static int staticMethod_Add(int x, int y) { return x + y; }
8     public static double staticMethod_Add(double x, double y) { return x + y; }
9     public static int staticMethod_Mul(int x, int y) { return x * y; }
10    public static double staticMethod_Mul(double x, double y) { return x * y; }
11    public int instanceMethod_Add(int x, int y) { return x + y; }
12    public double instanceMethod_Add(double x, double y) { return x + y; }
13    public int instanceMethod_Mul(int x, int y) { return x * y; }
14    public double instanceMethod_Mul(double x, double y) { return x * y; }
15 }
16
17 void main() {
18     new Subject().staticMethod_Add(1, 2);
19     new Subject().instanceMethod_Add(1, 2);
20     new Subject().staticMethod_Add(1, 2);
21     System.out.println(...);
22     // 2, 1
23 }
```

Task 2 (40%) – Example Use of Soot – Our Own Phase – Transformer

Implement: insert **what** code to **where**, with **what recorder**

- How to auto insert invocation of the var-incr function into CUT?
 - A customized **BodyTransformer** to insert **invocation statements** **before each return statement**.

```
public class Instrumenter extends BodyTransformer {  
    /* some internal fields */  
    static SootClass counterClass;  
    static SootMethod addStaticInvocationMethod,  
    // ...  
    @Override  
    protected void internalTransform(Body body,  
        // body's method  
        SootMethod method = body.getMethod();  
  
        // we dont instrument constructor (<ini  
        if (method.isConstructor() || method.is  
            return;  
        }  
  
        // debugging  
        System.out.println("instrumenting method: " + method.getSignature());  
  
        // get body's unit as a chain  
        Chain<Unit> units = body.getUnits();  
  
        // get a snapshot iterator of the unit since we are going to  
        // mutate the chain when iterating over it.  
        //  
        Iterator<?> stmtIt = units.snapshotIterator();  
    }  
}
```

=> Optimization phase registered to soot.

Java Bytecode → Jimple Code → Optimization on Jimple code → Java Bytecode (optimized)

Key method: **internalTransform**
It will be executed when each function in the CUT is analyzed.

```
// typical while loop for iterating over each statement  
while (stmtIt.hasNext()) {  
    // cast back to a statement.  
    Stmt stmt = (Stmt) stmtIt.next();  
  
    // there are many kinds of statements, here we are only  
    // interested in return statements  
    // NOTE: there are two kinds of return statements, with or without return value  
    if (stmt instanceof ReturnStmt || stmt instanceof ReturnVoidStmt) {  
        // now we reach the real instruction  
        // call Chain.insertBefore() to insert instructions.  
        //  
        // 1. first, make a new invoke expression  
        InvokeExpr incExpr = null;  
        if (method.isStatic()) {  
            // if current method is static, we add static method invocation counter  
            incExpr = Jimple.v().newStaticInvokeExpr(  
                addStaticInvocationMethod.makeRef(), IntConstant.v(1));  
        } else {  
            // if current method is instance method, we add instance method invocation counter  
            incExpr = Jimple.v().newInstanceInvokeExpr(  
                addInstanceInvocationMethod.makeRef(), IntConstant.v(1));  
        }  
  
        // 2. then, make a invoke statement  
        Stmt incStmt = Jimple.v().newInvokeStmt(incExpr);  
  
        // 3. insert new statement into the chain, before return statement  
        // (we are mutating the unit chain).  
        units.insertBefore(incStmt, stmt);  
    }  
}
```

How to identify the place to insert?
-- Find the "ReturnStmt"

How to make an invocation to the funcs?
-- Soot API 'newStaticInvokeExpr'.

How to insert? – Soot API
newInvokeStmt' and 'insertBefore'.

Task 2 (40%) – Example Use of Soot – Tool Entrypoint

- **An entry point for the tool to setup Soot, start instrumentation, run tests, & show res.**

- Overall Procedure:

Compile => Instrument => Run CUT on Tests => R

```
public class EntryPoint {  
    public static void main(String[] args) {  
        instrumentWithSoot();  
        // after instrument, we run Junit tests  
        runJunitTests();  
        // after junit test running, we have already get the counting in the Counter class  
        System.out.println("Invocation to static methods: " + Counter.getNumStaticInvocations());  
        System.out.println("Invocation to instance methods: " + Counter.getNumInstanceInvocations());  
    }  
}
```

- To Instrument:

- Add our phase *Instrumenter* into Soot
- Our customized phase will be executed by:

```
private static void instrumentWithSoot() {  
    // the path to the compiled Subject class file  
    String classUnderTestPath = "./raw-classes";  
    String targetPath = "./target/classes";  
  
    String classPathSeparator = ":";  
    if (System.getProperty("os.name").toLowerCase().contains("win")) {  
        classPathSeparator = ";";  
    }  
  
    /*Set the soot-classpath to include the helper class and class to analyze*/  
    Options.v().set_soot_classpath(Scene.v().defaultClassPath() + classPathSeparator + targetPath +  
  
    // we set the soot output dir to target/classes so that the instrumented class can override the  
    Options.v().set_output_dir(targetPath);  
  
    // retain line numbers  
    Options.v().set_keep_line_number(true);  
    // retain the original variable names  
    Options.v().setPhaseOption("jb", "use-original-names:true");  
  
    /* add a phase to transformer pack by call Pack.add */  
    Pack jtp = PackManager.v().getPack("jtp");  
  
    Instrumenter instrumenter = new Instrumenter();  
    jtp.add(new Transform("itp.instrumenter", instrumenter));  
  
    String classUnderTest = "castle.comp5111.example.Subject";  
    // pass arguments to soot  
    soot.Main.main(new String[]{classUnderTest}); // added phases will be executed in this method  
}
```

Task 2 (40%) – Example Use of Soot (cont.)

Check what a Jimple code looks like: - may help you understand & debug (original vs edited).

Foo.java

```
1 public class Foo {
2
3     static int factorial (int x){
4         int result = 1;
5         int i = 2;
6         while (i <= x) {
7             result *= i;
8             i++;
9         }
10        return result;
11    }
12 }
13 }
```

Foo.jimple

```
1 public class Foo extends java.lang.Object
2 {
3
4     public void <init>()
5     {
6         Foo r0;
7
8         r0 := @this: Foo;
9
10        specialinvoke r0.<java.lang.Object: void <init>()>();
11
12        return;
13    }
14
15    static int factorial(int)
16    {
17        int i0, i1, i2;
18
19        i0 := @parameter0: int;
20
21        i1 = 1;
22
23        i2 = 2;
24
25        label1:
26        if i2 > i0 goto label2;
27
28        i1 = i1 * i2;
29
30        i2 = i2 + 1;
31
32        goto label1;
33
34        label2:
35        return i1;
36    }
37 }
```

Commands for easy understanding

```
java soot.Main -f jimple Foo
```

Same as above, but produce Jimple in sootOutput/Foo.jimple.

```
java soot.Main Foo
```

Process Foo.class in the current directory and produce a new class file in sootOutput/Foo.class.

Task 2 (40%) – Requirements: Output Format & Runner Scripts

- Req 1: coverage report, including **at least** the following information:
 - The line coverage for CUT (*final percentage + raw counts of cov/tot lines*), *incl. inner classes*.
 - The coverage of each Java source code line (not Jimple line No. format: *Line No, covered or missed*).
 - Encouraged: the statements in each line (and their coverage).

comp5111.assignment.cut.Regression_6_Test - Line Coverage

Overview:

Overall:

total: (*omitted*) covered: (*omitted*) percentage: 55.2%

comp5111.assignment.cut.Subject\$DateTimeAlgorithms

total: 159 covered: 101 percentage: 63.5%

...(omitted. please list the coverage of all inner classes)...

Details:

comp5111.assignment.cut.Subject\$DateTimeAlgorithms

int daysBetweenDateStrings(java.lang.String,java.lang.String,

Covered Line 687 (*you are encouraged to print Jimple stmts in this line:* parts1 = virtualinvoke dateStr1.<java.lang.String: java.lang.String[] split(java.lang.String)>("-"))

Missing Line 691 (*you are encouraged to print Jimple stmts in this line:* \$stack14 = parts1[0]; y1 = staticinvoke <java.lang.Integer: int parseInt(java.lang.String)>(\$stack14))

...(omitted)...

Feb. 10, 2026

Task 2: Line coverage measurement using Soot (40%)

Soot is a framework for analyzing and transforming Java programs. We prepared a short introduction about Soot [here](#). Answers to some common questions are available [here](#).

For Task 2 you may refer to an [example test suite](#) along with its [line coverage](#) collected by our implementation

Req 2: Prepare a one-click script + instruction to use your tool.

```
for (( i = 0; i < 1; i++ )); do
```

```
arguments="$i comp5111.assignment.cut.Regression_"$ti"_Test "
```

```
arguments=$arguments'comp5111.assignment.cut.Subject comp5111.assignment.cut.Subject$stringAlgorithms  
comp5111.assignment.cut.Subject$DateTimeAlgorithms (omit some classes here. please complete.) '
```

```
mvn clean compile exec:java -Dexec.mainClass="comp5111.assignment.Assignment1" -Dexec.args="$arguments"  
done
```

(Example codes here are for reference. They may be incomplete and contain minor errors.)

Task 2 (40%): Measuring Test Coverage with Soot (cont.)

Kind Tips – How to start?

- **Start from the example project**
 - Understand the **code & logic** about **traversing statements** and **recording info**. Your assignment is ***quite similar*** to the example project.
 - **Example Task (counting executed return stmts)**
 - Insert the **statements to increase counter** for each return statement.
 - **Task 2 (line coverage)**
 - Insert the **code to record stmts & line nos** for each target Jimple statement.

Figure out what to record and how to implement them by yourself.

Task 2 (40%): Measuring Test Coverage with Soot (cont.)

Other Tips

- You can crosscheck your Soot-based results with not only TA-line.txt but also EclEmma. However, you may find the number of stmts counted by Soot is different from by EclEmma.
 - **Possible**, because EclEmma and Soot use different intermediate representation. You may only refer to the EclEmma stmt coverage **for reference** – but they are usually **close**.
- Record cover state of statement as well. It is helpful for second programming assignment. A major task in PA2 is also about calculating something based on instrumentation records.
- Others:
 - Go through the soot document to learn **other useful APIs** besides the ones in example project.
 - Read **FAQ** in our repository first; contact with TAs if still serious issues (but TAs won't help debug!).
 - Two Materials: <https://www.brics.dk/SootGuide/> , <https://noidsirius.medium.com/a-beginners-guide-to-static-program-analysis-using-soot-5aee14a878d>

Task 3 (35%): Explore the Use of GenAI for Test Generation

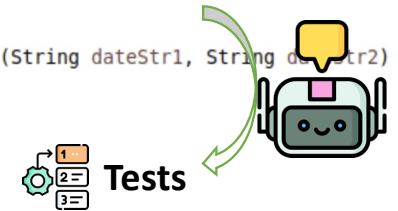
- Requirements: (Please refer to our very detailed instructions)

- The core topic:** Generate tests based on the **specification** (docstring, no code) of functions.

- Four subtasks.

- Test Generation Based on Current Specifications (10%):* Design effective prompts to make the LLM generate JUnit test cases for 10-20 functions based on only the function's docstring and signature.
- Analysis of Limitations of Current Specifications for Test Generation (10%):* Identify limitations of these tests in coverage, correctness, etc., analyze what issues are caused by the current docstring's content (e.g., missing certain important details, ambiguous descriptions, etc.)
- Suggestions for Specification Improvement (5%):* Propose 2-4 concrete ways to mitigate docstring issues.
- Justify Effectiveness of the Enhanced Specifications (10%):* Enhance the specifications of some functions based on your suggestions, and demonstrate its effectiveness with the improvement in test case quality.

```
/**  
 * Returns the number of days between two date strings in "yyyy-MM-dd" format.  
 *  
 * @param dateStr1 the first date string  
 * @param dateStr2 the second date string  
 * @return the number of days between the two dates, or Integer.MIN_VALUE if  
 *         invalid  
 */  
public static int daysBetweenDateStrings(String dateStr1, String dateStr2) {
```



Task 3 (35%): Explore the Use of GenAI for Test Generation (cont.)

- Write a report to discuss findings in these subtasks.

	Length (soft limits)
• T4.1 Test Generation Based on Current Specifications (10%).	– 100-250 words.
• T4.2 Analysis of Limitations of Current Specifications for Test Generation (10%).	– 200-350 words.
• T4.3 Suggestions for Specification Improvement (5%).	– 100-250 words.
• T4.4 Justify Effectiveness of the Enhanced Specifications (10%).	– 200-350 words.
- **Note:** Keep your report **informative** and **concise**. Concrete examples, intuitive visualizations (figures), and quantitative measurements (tables) are expected to support your analysis.
- We encourage the use of the free hkgai-v1-series models on HKUST OpenWebUI platform, but other publicly available LLMs are also allowed.
- **Submission:** A PDF, with the discussions for each subtask separated in different sections.

Submission:

- **Submission Requirement:**

- Submit via **Canvas**.
- Please check our course website & repo for the very detailed specification.
 - all info mentioned in this tutorial, as well as the detailed submission format requirement, hint and guidance, FAQs, etc.

- **Deadline:** 23:55, 15th March 2026

Do it as early as possible to ensure enough debug time & Try your best.

<Reading assignment time slot booking!>